

In system design, web servers and proxies are crucial components that facilitate seamless user-application communication. Web pages, images, or data are delivered by a web server in response to requests from clients, like browsers. A proxy, on the other hand, acts as a mediator between clients and servers, enhancing caching, and load distribution. When combined, they support the development of dependable, safe, and effective systems.

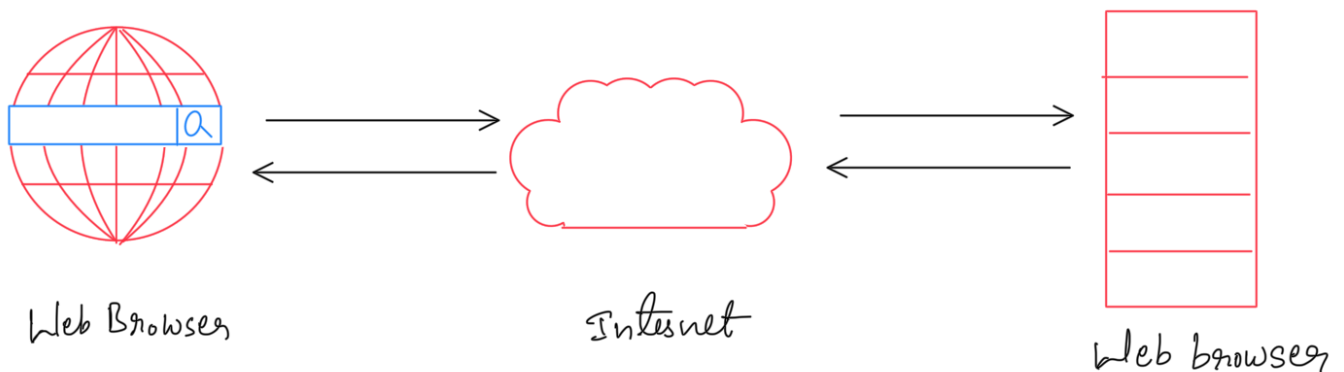
What are Webservers?

Every website sits on a computer that is known as a web server. Web server is a computer system that hosts websites and runs web server software. It is responsible for hosting websites. Some of the popular web servers these days are Apache HTTP and Microsoft IIS. Also, the machine(normal computer) to act as a web server should possess two traits:

- Machine should be able to run web server software.
- Machine should throughout be connected to the internet so as to serve clients being available all time.

Every web server connected to the internet is given a unique address of 4 digits that are separated by dots as shown below:

68.122.31.125



Types of Webservers

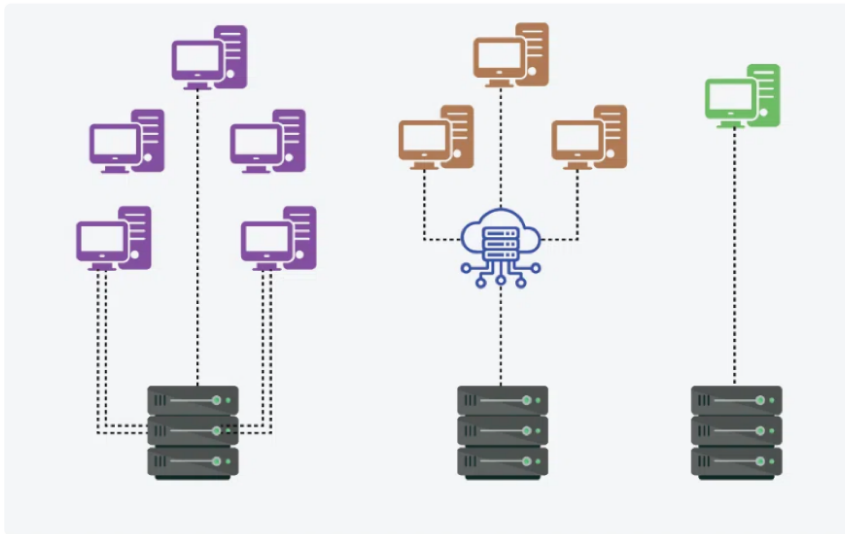
Web servers are categorized as per hosting websites into:

- **Shared Hosts:** Designed to host over 100+ websites.
- **Dedicated Hosts:** Designed to host only 1-2 websites
- **Custom Hosts:** Specifically designed as per internal requirements for any large-scale websites.

That's why personal websites or indeed generic small websites uses shared hosts while on the other hand large-scale organization that generates tremendous traffic load uses dedicated and

other hand large-scale organization that generates tremendous traffic load uses dedicated and even if there is a requirement for custom server modification then we do even use custom hosts.

Shared and Dedicated Hosts



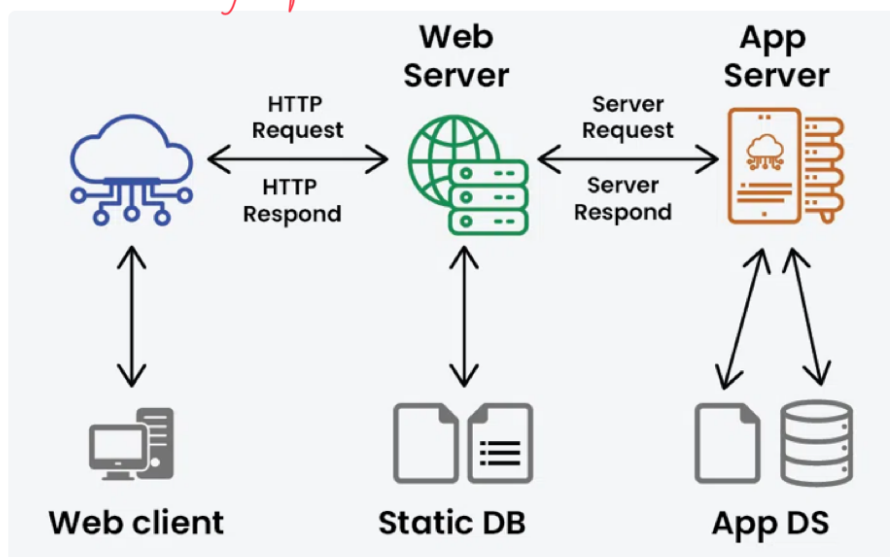
How web server Works?

When a web client (like a browser) sends a request to a web server, it uses an HTTP request to communicate. Here's what happens next:

- If the requested data is already in the web server's static database, the server sends an immediate response back to the client.
- If the data isn't there, the web server forwards the request (called a servlet request) to the application server to process it.

In this case, the application server looks in its datastore to find the required information, processes it, and then sends a response back to the web server. Finally, the web server sends this response to the client.

Working of Server



Application Server

Application server is specifically designed to run applications (generally invoking extreme computations) that includes both hardware and software to provide an environment to run the applications.

- **Usage:** Provides environment to run specific applications such as cloud applications and many more and do provides computational power.
- **Note:** Application server is only required when there is intense computation required which web server can not handle.

Role of Web Servers in System Design

- **Handling Client Requests:**
 - Web servers are in charge of accepting and handling incoming HTTP/HTTPS requests from clients, which are often browsers.
 - They operate as the first point of contact, decoding these inquiries and directing them to the relevant applications or resources.
- **Serving Static and Dynamic Content:**
 - Web servers deliver static content (such as HTML, CSS, and images) directly from their file system.
 - In order to build and serve dynamic content—which is produced instantly in response to user inputs or database queries—they also interface with application servers or frameworks.
- **Load Balancing:**
 - In larger systems, web servers can distribute incoming traffic across multiple backend servers.
 - This load balancing ensures that no single server becomes a bottleneck, thereby improving system reliability and performance.
- **Content Delivery and Caching:**
 - Web servers often cache frequently requested content to reduce response times and server load.
- **SSL/TLS Termination:**
 - Web servers manage SSL/TLS encryption to secure data transmission between clients and servers.
 - They handle the complex processes of encrypting and decrypting data, ensuring secure communications without burdening backend servers with these tasks.

Examples of Web Servers

Popular Web servers used are as follows:

- **Tomcat servers:** Tomcat is used for Java web applications that do not need to meet all of the standards of Java EE but still need a dependable solution. Tomcat isn't regarded as a full application server because it also functions as a Web server and a Servlet container.
- **Apache servers:** accept directory (HTTP) requests from users of the Internet and deliver the

- **Apache servers**, accept directory (HTTP) requests from users of the internet and deliver the desired data in the form of files and Web pages. The features of Apache are supported by a large portion of the software and code on the Internet.
- **Node.js servers**: It includes a built-in HTTP module that enables it to transmit data using the HTTP.

Note:

- *Apache HTTP is the most popular web server in the world.*
- *Apache with Tomcat Module is used to have JSP and J2EE support.*

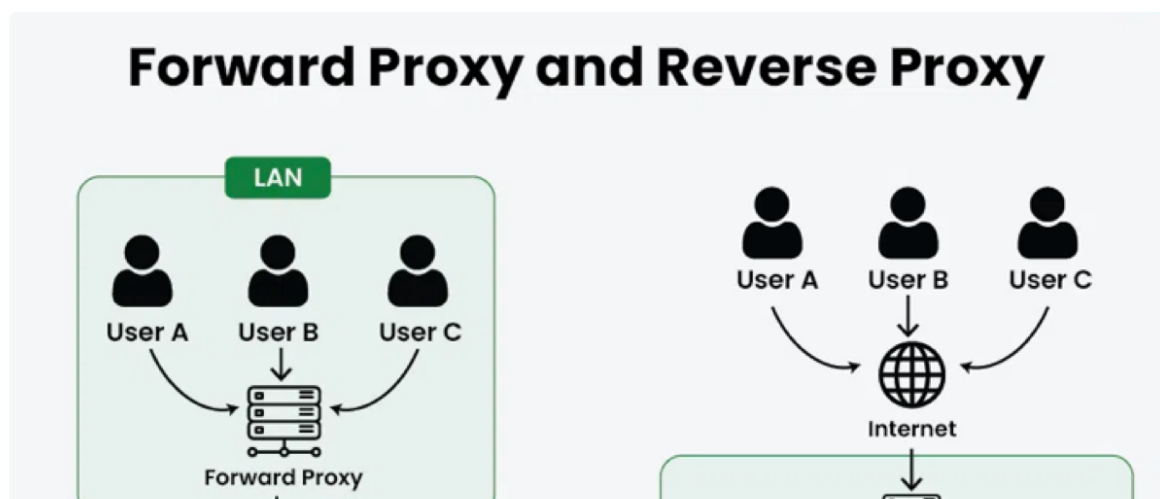
Role of Proxies in Designing Systems

By serving as a bridge between clients (such as browsers) and servers, proxies are crucial to the development of effective and secure systems. Here are a few significant ways in which they help:

- **Security**: By preventing the server from being directly exposed to the internet, proxies may hide the IP address of the real server, so enhancing security.
- **Load Balancing**: Incoming traffic can be split up across several servers by proxies. This enhances the system's overall speed and reliability, helps balance the demand, and keeps any one server from becoming overburdened.
- **Caching**: Proxies store copies of frequently requested data. When users ask for this data, the proxy can deliver it quickly from its cache, reducing the load on the main server and speeding up response times.
- **Access Control**: Proxies can control which requests reach the server, blocking or filtering out unwanted traffic.
- **Traffic Monitoring**: Proxies can log data on traffic patterns, providing insights for optimizing performance and enhancing security.

Types of Proxies

Proxies can sit on the client's local server side and also somewhere between the client and remote servers which we are going to discuss. Basically, they are of 2 types only because the proxy server can reside on the client's local server or anywhere between the client and the remote servers.





1. Forward Proxy

A server that serves as an intermediary between your computer or other device and the internet is known as a forward proxy. Your request passes through the forward proxy before reaching the website when you attempt to view it. On your behalf, the proxy then submits the request to the website, receives the response, and sends it back to you.

When to use a forward proxy server?

- Forward proxies are in handy when we don't want the main server to be able to see the client's IP because it only has the proxy server's IP.
- Used in scenarios where a large number of clients are present and the proxy server is handling all traffic.
- Used when we block a specific website.

2. Reverse Proxy

A reverse proxy is a server that sits between clients (like web browsers) and another server, handling requests on behalf of that server. Instead of a client connecting directly to the main server, the reverse proxy intercepts requests and forwards them to the server.

When to use a reverse proxy server?

- **Load balancing:** Distributes client requests in a certain manner across servers ensuring no server is overloaded leading to higher speed and capacity utilization.
- Reducing DDoS attacks. It is because only proxy server is accessible to the outer world where servers are not.
- Caching a server response.

Forward Proxy vs. Reverse Proxy

Aspect	Forward Proxy	Reverse Proxy
Purpose	Acts on behalf of the client, forwarding client requests to the server	Acts on behalf of the server, handling client requests on behalf of the server
Client Interaction	The client is aware of the forward proxy and explicitly configures it to communicate with the server	The client is usually unaware of the reverse proxy, interacting with it as if it were the actual server
Use Cases	Used to anonymize client requests, control internet access, and cache data locally	Used to load balance traffic, provide security features like SSL termination, and cache server responses

Caching	Caches data from the internet to reduce bandwidth usage and speed up access for clients	Caches data from servers to speed up response times for clients and reduce the load on backend servers
Protocol Translation	Can translate protocols (e.g., HTTP to HTTPS) for outgoing requests	Can handle SSL termination, translating HTTPS requests into HTTP for backend servers

Disadvantages of Proxy Server

Below are some of the disadvantages of proxy server:

- **Proxy Server Risks:** Free installation does not invest much in backend hardware or encryption. It will result in performance issues and potential data security issues. If you install a “free” proxy server, treat it very carefully, some of those might steal your credit card numbers.
- **Browsing history log:** The proxy server stores your original IP address and web request information is possibly unencrypted from and saved locally. Always check if your proxy server logs and saves that data.
- **No encryption:** No encryption means you are sending your requests as plain text. Anyone will be able to pull usernames and passwords and account information easily. Keep a check that the proxy provides full encryption whenever you use it.

Conclusion

In summary, web servers and proxies are essential tools in designing robust, efficient, and secure systems. Web servers handle client requests and deliver content, while proxies—both forward and reverse—help manage traffic, balance loads, enhance security, and improve speed through caching. Together, they create a seamless user experience and allow systems to scale and adapt to growing demands.